

Trabalho Prático: Código Secreto

1 Introdução

Neste trabalho, você deverá desenvolver um jogo em linguagem C inspirado no jogo **Código Secreto** (código mestre ou senha) que utiliza **cores**.

A ideia principal é simples: o programa deve gerar uma sequência secreta de cores, e o jogador deve tentar descobrir essa sequência dentro de um número limitado de tentativas. A Figura 1a mostra o tabuleiro de um jogo físico. Enquanto a Figura 1b mostra as regras do jogo.



(a) O jogo



(b) Regras do jogo

Figura 1: Jogo Código Secreto

2 Descrição do jogo

O programa deve gerar aleatoriamente uma sequência de cores. O jogador não verá essa sequência no início da partida.

A cada tentativa, o jogador informa uma sequência de cores com o mesmo tamanho da sequência secreta. Após cada tentativa, o programa deve informar ao jogador o resultado da jogada.

2.1 Cores disponíveis

O jogo deve trabalhar com uma lista fixa de cores. Por exemplo:

Código	Cor
1	Vermelho
2	Azul
3	Verde
4	Amarelo
5	Roxo
6	Laranja

Você poderá usar os nomes das cores, os códigos numéricos ou ambos. Porém, o programa deve deixar claro para o jogador quais opções estão disponíveis.

O jogo terá três níveis de dificuldade:

Nível	Tamanho da sequência	Número de tentativas
Fácil	4 cores	10
Médio	5 cores	12
Difícil	6 cores	15

3 Regras da partida

O programa deve possuir as características descritas abaixo.

3.1 Menu inicial

Ao executar o programa um menu com os seguintes comandos do jogo devem ser apresentados ao usuário, são eles:

X: Sair – encerrar o jogo. Caso tenha um jogo em andamento, pergunte ao usuário se quer salvar o jogo antes de sair.

N: Novo jogo – inicia um novo jogo e solicita do usuário o seu nome e o nível de dificuldade:

1: nível fácil

2: nível médio

3: nível difícil

C: Carregar jogo – solicita o nome do arquivo texto contendo o jogo e apresenta o jogo no estado armazenado

R: Ranking – o programa deve imprimir uma lista com o ranking dos jogadores.

S: Salvar – armazena em um arquivo o jogo em seu estado atual. Solicita do usuário o nome do arquivo. O usuário não precisa digitar a extensão, o programa deve inserí-la (.cor) automaticamente.

A: Ajuda – exibe as instruções de jogo e como usar o programa.

Importante: seu programa deve proibir que o usuário execute comandos inválidos. O usuário deve ser alertado com uma mensagem de erro caso digite um valor inválido. O programa também deve detectar quando o jogo acaba com vitória do jogador.

3.2 Geração da sequência secreta

Ao iniciar um partida, o programa deve:

- Solicitar o nome;
- Perguntar o nível de dificuldade;
- Definir o tamanho da sequência e o número máximo de tentativas;
- Alocar dinamicamente os vetores necessários;
- Gerar aleatoriamente a sequência secreta de cores.

A essa sequência não deve ser mostrada ao jogador durante a partida.

3.3 Entrada das tentativas

Em cada tentativa, o jogador deve informar uma sequência de cores. Exemplo:

Tentativa 1 de 10 (Digite 0 para voltar ao menu)
Digite 4 cores usando os códigos de 1 a 6: **1 2 3 4**

O programa deve validar se os valores digitados são válidos. Por exemplo, se o jogador digitar:

1 2 9 4

O valor 9 é inválido, pois não existe uma cor com esse código. Nesse caso, o programa deve avisar o erro e pedir novamente a tentativa.

Durante a entrada de dados o jogador pode voltar ao menu principal, para isso deve digitar apenas 0. No entanto, você deve confirmar se ele deseja voltar, veja o exemplo:

Tentativa 1 de 10 (Digite 0 para voltar ao menu)
Digite 4 cores usando os códigos de 1 a 6: **0**
Deseja voltar ao menu principal (S/N):

3.4 Comparação da tentativa com a sequência secreta

Após cada tentativa válida, o programa deve comparar a jogada do jogador com a sequência secreta.

O programa deve informar:

- Quantas cores estão corretas na posição correta;
- Quantas cores existem na sequência secreta, mas estão em posição errada.

Exemplo:

Sequência secreta:

Vermelho - Azul - Verde - Amarelo

Tentativa do jogador:

Vermelho - Verde - Azul - Roxo

Resultado do programa:

Vermelho - Verde - Azul - Roxo [- E E C]

Explicação:

- Vermelho está correto e na posição correta [C];
- Verde existe na sequência, mas está em outra posição [E];
- Azul existe na sequência, mas está em outra posição [E];
- Roxo não existe na sequência secreta [-].

Atenção: A resposta com a indicação de acertos **NÃO DEVE** ser na mesma posição da sequência da entrada.

3.5 Condição de vitória e derrota

O jogador vence quando acertar todas as cores na posição correta.

Exemplo:

Parabéns! Você descobriu a sequência secreta!

O jogador perde quando acabar o número de tentativas.

Exemplo:

Fim de jogo! Você não conseguiu descobrir a sequência.

A sequência correta era:

Vermelho - Azul - Verde - Amarelo

4 Requisitos obrigatórios do programa

O objetivo do trabalho é praticar os principais conceitos estudados na disciplina de Introdução à Programação, incluindo:

- Entrada e saída de dados;
- Estruturas condicionais `if`, `else` e `switch`;
- Laços de repetição `for`, `while` e `do-while`;
- Funções;

- Estruturas homogêneas: vetores e matrizes;
- Estruturas heterogêneas: **struct**;
- Alocação dinâmica de memória: **malloc**, **calloc** e **free**;
- Manipulação de arquivos.

1. **Alocação Dinâmica:** Utilize vetores e matrizes para armazenar a sequência secreta, a jogada atual, o histórico de jogadas, etc. Como o jogo pode ter dimensões variadas (4×10 , 5×12 ou 6×15). A memória para essas informações deve ser alocada dinamicamente (**malloc()**/**calloc()**) e liberada (**free()**) corretamente.
2. **Uso de Structs:** O estado do jogo não pode ficar “solto” em variáveis dispersas. Você deve criar uma **struct** que encapsule as principais informações do jogo, como o nome do jogador o nível, o número máximo de tentativas, etc.
3. **Modularização:** O código deve ser dividido em funções. É **proibido** o uso de variáveis globais. Somente tipos e constantes podem ser globais. Toda a comunicação entre funções deve ser via passagem de parâmetros.
4. **Persistência:** O jogo deve ser capaz de salvar o estado atual em arquivo e carregar jogos anteriores.

5 Arquivo de ranking

O jogo deve permitir armazenar os resultados de até 10 jogadores mostrando-os ao ser selecionada a opção correspondente no menu inicial, para isso as seguintes funcionalidades devem ser implementadas ao jogo:

- Manter um arquivo binário com o ranking (denominado **ranking.rnk**) que permita ler e gravar as informações dos jogadores vencedores;
- Sempre que uma partida finalizar, mostrar o o número de tentativas do jogador e a posição no ranking. Caso ele esteja entre os 10 primeiros;
 - o ranking dever estar ordenado do menor número de tentativas para o maior. Se o número de tentativas for igual utilize como critério de desempate o nível. Ordenando do difícil para o fácil
- Sempre mantenha o arquivo de ranking atualizado.

Utilize a seguinte **struct** para gravar as informações no arquivo.

```

1 typedef struct {
2     char nome[51];
3     int nivel;
4     int tentativas;
5 } Ranking;
```

6 Arquivo do jogo

Ao selecionar a opção “Salvar”, o programa deve armazenar as seguintes informações da partida em um arquivo de texto:

- Linha 1: Nome do jogador;
- Linha 2: Nível escolhido;
- Linha 3: Sequência correta;
- Linha 4: Número de tentativas jogadas;
- Linha 5 em diante: Histórico das tentativas.

```

1 Ana
2 F
3 1 2 3 4
4 3
5 1 3 2 5
6 1 2 3 5
7 4 1 2 3

```

O arquivo acima apresenta um jogo com a jogadora Ana no nível fácil (F). A sequência secreta do jogo é [Vermelho - Azul - Verde - Roxo]. Ana já fez 3 tentativas:

```

Vermelho Verde Azul Roxo
Vermelho Azul Verde Roxo
Vermelho Azul Verde Amarelo

```

7 Instruções gerais

- O trabalho é individual.
- O problema deve ser resolvido por meio de um programa em C.
- Inclua seu nome e número de matrícula como comentário em todos os arquivos .c e .h gerados.
- Você deverá entregar o código-fonte. Se forem vários arquivos compacte em formato ZIP.
- A entrega deve ser feita pelo Moodle.
- Após a entrega dos trabalhos serão marcadas entrevistas com cada um dos alunos para apresentação dos mesmos para os professores. Elas serão realizadas nos horários da disciplina.
- Não serão aceitos trabalhos que caracterizem cópia (mesma estrutura e algumas pequenas modificações) de outro ou feitos por IA.

8 Avaliação

O trabalho será avaliado da seguinte forma:

9 Exemplo de Execução

A seguir é exibido um exemplo, como referência, não é necessário segui-lo. Faça sua interface como achar mais interessante.

Você pode (e deve) customizar e melhorar as saídas do programa. A seguir segue um exemplo simples apenas para entendimento (os dados digitados pelo usuário estão destacados em azul):

Exemplo: Considere que a sequência secreta é: **vermelho, azul, amarelo, verde** (1, 2, 4, 3)

./codigo

=====

JOGO CÓDIGO SECRETO DE CORES

=====

Opções de jogo:

A - Ajuda
N - Novo Jogo
C - Carregar Jogo
S - Salvar Jogo
R - Ranking
R - Sair

Digite a opção: **N**

Digite seu nome: **Ana**

Escolha o nível:

1 - Fácil (4 cores, 10 tentativas)
2 - Médio (5 cores, 12 tentativas)
3 - Difícil (6 cores, 15 tentativas)

Opção: **1**

Cores disponíveis:

1 - Vermelho
2 - Azul
3 - Verde
4 - Amarelo
5 - Roxo
6 - Laranja

Tentativa 1 de 10

Digite 4 cores (zero): **5 2 3 4**

Resultado:

Rodada 1: 5 2 3 4 (E C C -)

Tentativa 2 de 10

Digite 4 cores: **1 2 4 3**

Resultado:

Rodada 1: 5 2 3 4 (E C C -)

Rodada 2: 1 2 4 3 (C C C C)

Parabéns, Ana! Você venceu!

Pressione uma tecla para o voltar ao menu principal!

10 Observações finais

Este trabalho deve ser desenvolvido com atenção à organização do código.

Um bom programa não é apenas aquele que funciona. Ele também deve ser claro, bem dividido em funções, comentado e fácil de entender.

Durante o desenvolvimento, teste o programa aos poucos:

- Primeiro, faça o menu funcionar;
- Depois, configure os níveis;
- Depois, gere a sequência aleatória;
- Depois, leia uma tentativa;
- Depois, implemente a comparação;
- Depois, adicione o histórico;
- Por fim, salve os dados em arquivo.

Não deixe para implementar tudo de uma vez. Desenvolva e teste por partes.